Python Cheatsheet

MATH60033A · Session 06 · Panel data and interactions

Thierry Warin, PhD · HEC Montréal · A2026

Everything you need to type, in one place. The first three sections are the same every week; the last one is the Python from session 5.

Starting Python in VS Codium

Open the course folder once — **File > Open Folder**, then pick the repository. Everything below assumes you are inside it.

Open a terminal with **View > Terminal** (`Ctrl` + `` ` `` on Windows and Linux, `Cmd` + `` ` `` on macOS). Create the environment the first time only:

```
python3 -m venv .venv      # macOS
python3 -m venv .venv      # Linux
py -m venv .venv           # Windows (PowerShell)
```

Then activate it — **every time you open a new terminal**:

```
source .venv/bin/activate  # macOS
source .venv/bin/activate  # Linux
.venv\Scripts\activate     # Windows (PowerShell)
```

You will see `(.venv)` at the start of the prompt. If you do not, nothing below will find pandas. Install the packages once, with the environment active:

```
pip install -r requirements.txt
```

Finally tell the editor which Python to use: `Ctrl`/`Cmd` + `Shift` + `P`, type **Python: Select Interpreter**, and choose the one inside `.venv`.

A new file, and running it

File > New File, then save it with a `.py` ending — `session03.py`, say. Write your code, save it, and in the terminal:

```
python session03.py        # macOS, Linux and Windows alike, inside .venv
```

Inside the environment the command is `python` on all three platforms. Outside it, macOS and Linux need `python3` and Windows needs `py`, which is one more reason to activate first.

For trying one line at a time, type `python` on its own to get the `>>>` prompt, and `exit()` to leave. The editor's **Run** triangle does the same thing as the command above.

The four things you always do

```
import pandas as pd

# 1 - build a DataFrame
df = pd.DataFrame({"country": ["FR", "DE", "IT"],
                   "gdp_pc_eur": [37_000, 46_000, 32_000]})

# 2 - look at what is in it
df.head()          # the first five rows
df.shape           # (rows, columns)
df.dtypes          # what type each column holds
```

```
df.info()          # types, and how many values are missing
df.describe()      # count, mean, sd and quartiles, per numeric column

# 3 - read a file in
df = pd.read_csv("data.csv")
df = pd.read_excel("data.xlsx")                    # needs openpyxl
df = pd.read_parquet("data/spine/core.parquet")

import qmib
df = qmib.load("core")                             # a course dataset, by name

# 4 - write a CSV out
df.to_csv("out.csv", index=False)   # index=False, or you get a stray column
```

From session 5

Load and shape

```
from sklearn.preprocessing import StandardScaler
Z = StandardScaler().fit(Xtr).transform(Xtr)   # fit on TRAIN only

pipe = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
# so the scaler never sees the held-out fold
```

Fit

```
Ridge(alpha=1.0).fit(Ztr, ytr)     # Lasso, ElasticNet take the same shape
```

Read the fit

```
model.coef_, model.alpha_          # scikit-learn's spelling of .params
```

Choose and validate

```
LassoCV(cv=5).fit(Ztr, ytr).alpha_     # lambda by cross-validation

alphas, coefs, _ = lasso_path(Z, y)    # the whole coefficient path

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=7)

KFold(n_splits=5, shuffle=True, random_state=7)

mean_squared_error(yte, model.predict(Zte)) ** 0.5    # RMSE

alphas = np.logspace(-3, 2, 100)   # a grid even in log-lambda
```